

Database Normalization

First Normal Form (1NF)

Hospital Patient Visits — Assessment Problem

Course: Database Systems | Lab Manual — Task 8

Topic: UNF → 1NF | XAMPP / MySQL 8.x

Date: 2026

UNF

→

1NF ✓

Database normalization is the process of organizing a relational database to reduce data redundancy and improve data integrity. It involves decomposing tables to eliminate undesirable characteristics like insertion, update, and deletion anomalies. This report focuses on the **First Normal Form (1NF)** — the very first step in the normalization journey.

1NF Rule (Plain English)		
■	Rule 1	Every cell must contain a single, atomic (indivisible) value.
■	Rule 2	No repeating groups or multi-valued attributes allowed.
■	Rule 3	Every row must be uniquely identifiable (primary key must exist).
■	Rule 4	All values in a column must be of the same data type.

2. Dataset — Hospital Patient Visits (Task 8)

The hospital currently records every consultation in a single unnormalized table. The raw data below contains information about patient visits, doctors, departments, and diagnoses all crammed into one flat structure. Our goal is to bring this data into **First Normal Form**.

BEFORE: Unnormalized Table (UNF)

This is the ORIGINAL unnormalized data — multiple facts per row, redundancy everywhere.

VisitDate	PatientID	PatientName	PatientPhone	DoctorID	DoctorName	Specialty	DeptName	DeptHead	Diagnosis
2026-04-10	P-201	Hassan	0300-1112233	D-30	Dr. Imran	Cardiology	Heart Care	Dr. Tariq	Hypertension
2026-04-10	P-202	Mehreen	0301-4445566	D-31	Dr. Asma	Dermatology	Skin Clinic	Dr. Asma	Eczema
2026-04-11	P-201	Hassan	0300-1112233	D-31	Dr. Asma	Dermatology	Skin Clinic	Dr. Asma	Allergy
2026-04-12	P-203	Junaid	0302-7778899	D-30	Dr. Imran	Cardiology	Heart Care	Dr. Tariq	Arrhythmia

■ Red highlighted cells = redundant/repeated data | Rows 3 & 4 show P-201 (Hassan) duplicated | Doctor & Department info repeats for every visit

3. Functional Dependencies & Anomalies in UNF

3.1 Identified Functional Dependencies

Functional Dependency	Attributes	Notes
VisitID →	VisitDate, PatientID, DoctorID, Diagnosis, Fee	Direct visit attributes
PatientID →	PatientName, PatientPhone	Patient info depends on PatientID
DoctorID →	DoctorName, Specialty, DeptName, DeptHead	Doctor/Dept info depends on DoctorID
DeptName →	DeptHead	Transitive: dept determines its head (3NF issue)

3.2 Anomalies in the Unnormalized Table

Anomaly Type	Description	Example from Bookstore Data
Insertion Anomaly	Cannot add a new doctor/department until they have at least one patient visit.	If Dr. Zara (Neurology) is hired, we CANNOT record her info until a patient visits her.
Update Anomaly	Changing one fact requires updating multiple rows.	If Dr. Imran moves to office R-40, ALL rows with D-30 must be updated separately.
Deletion Anomaly	Deleting a row accidentally removes unrelated facts.	If we delete Mehreen's visit (V-9002), we lose all info about Dr. Asma if she has no other visits.

Step 1	Check for Multi-Valued Cells The UNF table already has single values per cell (unlike the bookstore example). However, the table still violates 1NF because ALL columns are in one massive flat table with no clear primary key defined. We must identify the candidate key.
Step 2	Identify the Primary Key Since each row represents one patient visit, VisitID uniquely identifies each row. VisitID → all other attributes. This becomes our Primary Key for the 1NF table.
Step 3	Ensure Atomicity Check every column: VisitDate, PatientName, PatientPhone, DoctorName, Specialty, DeptName, DeptHead, Diagnosis, Fee — all contain single atomic values. ■
Step 4	Create the 1NF Table We keep all columns in ONE table but now formally declare the primary key (VisitID) and ensure every cell holds exactly one value. The table is now in 1NF. Note: redundancy still exists (patient & doctor info repeats) — that is fixed in 2NF & 3NF.

5. AFTER: First Normal Form (1NF) Table

■ All cells are atomic | No repeating groups | Primary Key = VisitID | Every row uniquely identified

Table: HospitalVisit_1NF

ID	VisitDate	PatientID	PatientName	PatientPhone	DoctorID	DoctorName	Specialty	DeptName	DeptHead	Diagnosis
1	2026-04-10	P-201	Hassan	0300-1112233	D-30	Dr. Imran	Cardiology	Heart Care	Dr. Tariq	Hypertension
2	2026-04-10	P-202	Mehreen	0301-4445566	D-31	Dr. Asma	Dermatology	Skin Clinic	Dr. Asma	Eczema
3	2026-04-11	P-201	Hassan	0300-1112233	D-31	Dr. Asma	Dermatology	Skin Clinic	Dr. Asma	Allergy
4	2026-04-12	P-203	Junaid	0302-7778899	D-30	Dr. Imran	Cardiology	Heart Care	Dr. Tariq	Arrhythmia

■ Green column = Primary Key (VisitID) | * = Primary Key | Every cell holds exactly one atomic value | Table has 4 rows, 12 columns

6. Side-by-Side Comparison: UNF vs 1NF

Feature	UNF (Before)	1NF (After)
Primary Key	Not defined	VisitID (explicitly declared)
Atomic Values	■ Not guaranteed	■ All values are atomic
Repeating Groups	■ Possible	■ None
Row Identification	■ No formal PK	■ VisitID uniquely identifies each row
No. of Tables	1 (bloated flat table)	1 (same, but formally correct)
Redundancy	■■ High (patient & doctor repeat)	■■ Still present (fixed in 2NF)
Anomalies Fixed	■ All present	■■ Partially — atomicity resolved

The following SQL code creates the 1NF table in MySQL (XAMPP / phpMyAdmin). Run this in MySQL Workbench or the XAMPP phpMyAdmin SQL tab.

Step 1: Create Database & Use It

```
-- ===== -- DATABASE NORMALIZATION
-- 1NF -- Hospital Patient Visits (Assessment Problem — Task 8) --
===== -- Step 1: Create and select
the database CREATE DATABASE IF NOT EXISTS hospital_normalization; USE
hospital_normalization;
```

Step 2: Create the 1NF Table

```
-- ===== -- 1NF: HospitalVisit_1NF
-- Rule: Atomic values, no repeating groups, VisitID = PK --
===== DROP TABLE IF EXISTS
HospitalVisit_1NF; CREATE TABLE HospitalVisit_1NF ( VisitID VARCHAR(10) NOT NULL, --
PRIMARY KEY VisitDate DATE NOT NULL, PatientID VARCHAR(10) NOT NULL, PatientName
VARCHAR(50) NOT NULL, PatientPhone VARCHAR(20) NOT NULL, DoctorID VARCHAR(10) NOT NULL,
DoctorName VARCHAR(50) NOT NULL, Specialty VARCHAR(50) NOT NULL, DeptName VARCHAR(50) NOT
NULL, DeptHead VARCHAR(50) NOT NULL, Diagnosis VARCHAR(100) NOT NULL, Fee INT NOT NULL,
PRIMARY KEY (VisitID) );
```

Step 3: Insert All Data

```
-- ===== -- INSERT all 4 visit
records into 1NF table -- =====
INSERT INTO HospitalVisit_1NF VALUES ('V-9001', '2026-04-10', 'P-201', 'Hassan',
'0300-1112233', 'D-30', 'Dr. Imran', 'Cardiology', 'Heart Care', 'Dr. Tariq',
'Hypertension', 2500), ('V-9002', '2026-04-10', 'P-202', 'Mehreen', '0301-4445566',
'D-31', 'Dr. Asma', 'Dermatology', 'Skin Clinic', 'Dr. Asma', 'Eczema', 2000), ('V-9003',
'2026-04-11', 'P-201', 'Hassan', '0300-1112233', 'D-31', 'Dr. Asma', 'Dermatology', 'Skin
```

```
Clinic', 'Dr. Asma', 'Allergy', 2000), ('V-9004', '2026-04-12', 'P-203', 'Junaid', '0302-7778899', 'D-30', 'Dr. Imran', 'Cardiology', 'Heart Care', 'Dr. Tariq', 'Arrhythmia', 3000);
```

Step 4: Verify — SELECT All Data

```
-- ===== -- VERIFICATION: View the 1NF table -- ===== SELECT * FROM HospitalVisit_1NF; -- Expected: 4 rows, 12 columns -- VisitID is the Primary Key -- each row uniquely identified -- All values are atomic (one value per cell) -- Check specific patient visits SELECT VisitID, PatientName, Diagnosis, Fee FROM HospitalVisit_1NF ORDER BY VisitDate; -- Check for any remaining multi-values (should return 0) SELECT COUNT(*) AS TotalRows FROM HospitalVisit_1NF; -- Expected result: 4
```

8. Expected Query Output After 1NF

tID	VisitDate	PatientID	PatientName	PatientPhone	DoctorID	DoctorName	Specialty	DeptName	DeptHead	Diagnosis	F
01	2026-04-10	P-201	Hassan	0300-1112233	D-30	Dr. Imran	Cardiology	Heart Care	Dr. Tariq	Hypertension	2
02	2026-04-10	P-202	Mehreen	0301-4445566	D-31	Dr. Asma	Dermatology	Skin Clinic	Dr. Asma	Eczema	2
03	2026-04-11	P-201	Hassan	0300-1112233	D-31	Dr. Asma	Dermatology	Skin Clinic	Dr. Asma	Allergy	2
04	2026-04-12	P-203	Junaid	0302-7778899	D-30	Dr. Imran	Cardiology	Heart Care	Dr. Tariq	Arrhythmia	3

After running SELECT * FROM HospitalVisit_1NF; you should see the following output:

9. 1NF Compliance Checklist

■	Atomic Values	Every cell holds exactly one value — no comma-separated lists.
■	No Repeating Groups	No multi-valued attributes anywhere in the table.
■	Primary Key Defined	VisitID is declared as PRIMARY KEY.
■	Unique Rows	Each of the 4 rows is uniquely identifiable via VisitID.
■	Consistent Data Types	Each column stores values of a single type throughout.
■ ■	Redundancy Eliminated	Not yet — PatientName/Phone repeat for P-201. Fixed in 2NF.
■ ■	All Anomalies Fixed	Not yet — insertion/update/deletion anomalies remain. Fixed in 2NF/3NF.

10. Summary & What Comes Next

The Hospital Patient Visits table has been successfully converted to **First Normal Form (1NF)**. We now have a single table *Hospital/Visit_1NF* with 12 columns and 4 rows where every cell contains exactly one atomic value and VisitID serves as the primary key.

Status	Step	What Happens
■ Done	UNF → 1NF	Atomic values, PK defined, no repeating groups
■ Next	1NF → 2NF	Remove partial dependencies → split into Patient, Doctor/Dept, Visit tables
■ Later	2NF → 3NF	Remove transitive dependencies → separate Department from Doctor table